

A Backpressure Mitigation Scheme in Distributed Stream Processing Engines

Muhammad Hanif, Hyeongdeok Yoon, Choonhwa Lee

Division of Computer Science and Engineering, Hanyang University, Seoul, Republic of Korea
{honeykhan, ynoohel, lee}@hanyang.ac.kr

Abstract—There is a significant rise in the adaptation of streaming applications in the past decade by individuals researchers and organizations in both industry and academia. These applications are all based on the modern data stream processing systems that implement resource allocation and management in order to provide an uninterrupted track of queries over incoming input distributed data streams. More than a few stream processing engines exist to handle these distributed streaming applications. These distributed applications have open challenges like backpressure. In this paper, we introduce a backpressure mitigation mechanism for the distributed stream processing systems. The proposed backpressure mitigation technique is a generic one and is feasible to be implemented on top of a number of popular streaming frameworks. We use Flink as a testbed for this work and use its available APIs.

Keywords—Streaming, backpressure, SLA, Distributed Computing, Cloud Computing

I. INTRODUCTION

Big data, its streaming, moreover, its processing has increased in the past decade to the level of high magnetism due to its impacts on the wide range of applications such as malfunction detection in the field of heavy machine industry, real-time traffic analysis in urban population, social media and networking, intelligent online advertisement placement, real-time stock exchange trading analysis and so on. These applications pertain to processing of enormous volume, and continuous unbounded data such as text and transactional data; digital audio, video and image data; instant messages; network packet traces; and sensor data; for purposes such as filtering, aggregation, interdependent, and correlation. To process huge volume of data in fast and plane manner, many streaming computation engines have emerged. There are now several state of the art stream processing engines that are widely used by both academia and industry, including Apache Storm [1], Apache Spark [2], Apache Samza [3], Google Cloud Dataflow [4], IBM Stream[5], Apache Flink [6], and so on. These streaming engines have bigger community size, number of contributors, pull requests, size of its industrial community, and commit frequency at the source repositories are higher than all the other streaming processing systems out there.

With the increase in the popularity of these streaming data processing system (SDPS), there is an increase in the development of distributed streaming applications, such as

social media networking sites, online gaming, in-application purchasing, and so on. The use of SDPS in online social networking or online gaming require fast processing of large scale data feeds from multiple sources. To monitor the user feeds, windowed joins and windowed aggregations operations are used. One of the most popular use-cases is the tracking of in-application purchases while keeping distribution channel and product item under consideration. Another use case can be monitoring the fraud detection in the online banking and credit card system to make it work without a glitch and within the SLA agreement of the financial corporation. Such distributed stream processing applications have intermediate data streams for exchange of data between the operators. Intermediate data-stream is a logical abstraction of the data produced by an operator (such as Source operator) and can be used by one or more operators (such as Map, FlatMap, Aggregate operators) [7]. The data exchange data between concurrently running producers and consumer are often considered as pipelines execution which is done through pipelined intermediate streams.

In the case of stream processing, sometimes the stream pipeline produces data faster than the downstream operators has the ability to consume, this phenomenon is called backpressure. Records in job execution pipeline flow from upstream operators to downstream operators (such as from source operators to sink operators), and backpressure is propagated in the opposite direction i.e. upstream. This situation applies to numerous use-case scenarios in real-world applications and affects the performance of applications as well as the optimized resource utilization.

To overcome such problems and situations, we proposed a backpressure mitigation mechanism which checks the backpressure using a threshold and tries to mitigate it on the fly. The backpressure check mechanism investigates whether the backpressure has reached a threshold if so, it calls the backpressure mitigation module to handle it accordingly. We use apache Flink as testbed for this work.

The rest of the paper is organized as follows. Section II gives an overview of the background, section III briefly discuss the details of the backpressure mitigation mechanism. Section IV presents a use-case scenario and evaluation of the proposed work, and section V presents the concluding remarks of the paper.

This research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF), funded by the Ministry of Science and ICT (No.2017R1A2B4010395).

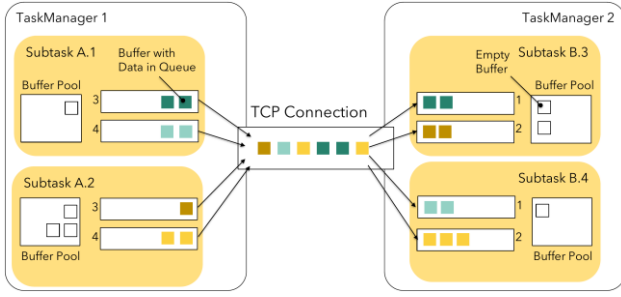


Fig. 1. Transfer of data between two tasks in Apache Flink [9].

II. BACKGROUND: APACHE FLINK

Apache Flink [6] was developed at TU Berlin as an open-source big data processing system. At its core, Flink is a distributed stream processing engine, running the program as directed acyclic graph (DAG) of operators connected with the incoming data streams using Flink Runtime environment. It supports both batch processing and streaming processing application. Flink considers the batch processing as a special case of streaming processing as opposed to apache Spark Streaming [8] which consider stream as a special case of batch processing as micro-batch processing. Flink provides its own memory management to avoid long-running JVM's garbage collector delays through memory serializations into segments. The exchange of data in a distributed environment occurs through buffers.

As Fig.1 shows, the producer takes a buffer from the pool, fills it up with data, then the consumer frees the buffer after receiving the data and inform the memory [9]. Flink provides a wide range of high-level state management APIs which handle stateful operators and their related operations. Managing the memory, incremental state updates, or check-pointing with big states are automatically performed transparently to users [10]. Flink has a valuable feature set for building and evaluating windows on stream of data. It supports custom window operators with a wide range of pre-defined windowing operators. It also has the support for out of order streams. It uses blocking queues to handle backpressure. Upon detection of congestion, the information is automatically transferred to upstream operators at minimal cost.

III. PROPOSED SYSTEM

We first throw the big picture of backpressure and blocking data exchange in stream processing systems. When the incoming streaming pipeline produces data faster than the downstream operators can consume, it causes backpressure. *Pipelined intermediate streams* exchange data between concurrently running producer operators and consumer operators resulting in pipelined execution. As a result, pipelined streams propagate backpressure from the consumer operators to producer operators, modulo some elasticity via intermediate buffer pools, in order to compensate for short-

```

1: procedure backpressureCheck (Stack stack)
2: backwardOp : backward operator instance
3: op : operator instance
4: task : task instance in the system
5: backpressureThreshold : defined threshold limit for backpressure
6: while stack is not empty do
7:   backwardOp ← stack.pop()
8:   op ← stack.top()
9:   for each task in op do
10:    if task.backpressure > backpressureThreshold then
11:      return backwardOp
12:    break
13:   end for
14: end while
15: end procedure

```

Fig. 2. backpressureCheck module: check the backpressure of each task with the defined threshold.

term throughput fluctuations. Backpressure is usually the after-effect of starting bottleneck tasks and propagating it backwards through the DAG via queues and TCP connections [11]. To avoid such backpressure propagation, we present backpressure mitigation mechanism, which checks the backpressure using a threshold and tries to mitigate it on the fly. Our proposed system is composed of two main modules as *backpressureCheck* module and *backpressureMitigate* module. The backpressure check mechanism investigates whether the backpressure has reached a threshold if so, it calls the backpressure mitigation module to handle it accordingly.

A. Backpressure Checking

In order to check the backpressure of the system, the *backpressureCheck* module periodically as shown in Fig. 2. The backpressureCheck module takes a stack as an argument and checks until the stack is empty. The algorithm first pops operator to the backwardOp and start from the top to check the backpressure for each operator. If the backpressure exceeds the threshold, it returns the backwardOp and returns.

B. Backpressure Mitigation

This module gets the backward operator as an argument and repeats the procedure for each task in the operator as shown in Fig. 3. It first locates the node for the task as in line

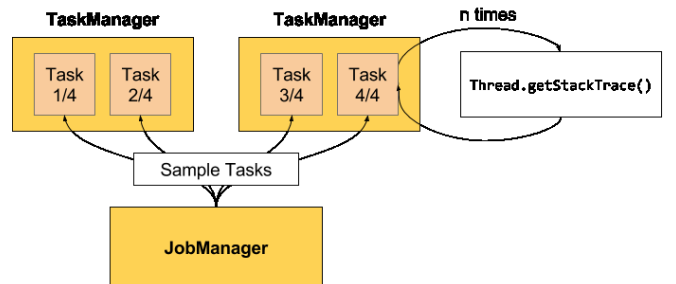


Fig. 4. Use-case Scenario: Sampling threads to monitor the task tracker by the job manager.

ten of the procedure. Then in order to find the backpressure for the node, it checks whether the node's CPU usage is beyond the threshold or not. Then it finds the in and outbound usage and compares it with the inbound and outbound threshold consecutively. If the node's inbound or outbound usage exceeds the defined threshold, it then decrements the operator's parallelism by value one. In case of the inbound usage only exceeds the threshold, it decrements its parallelism by value one (line 18). In case the outbound usage only exceeds the threshold, it decrements its parallelism by value one (line 22). Otherwise it increments the parallelism of backward operator and makes the new schedule available accordingly. Finally, an alert is issued to the user of the program about the backpressure situations before returning from the procedure.

IV. EVALUATION

In this section, we first present a use-case scenario where backpressure occurs. The job manager triggers calls periodically the *getStackTrack()* method of the *Thread* for each task of the job. This stack traces the samples of running tasks to monitor the back pressure [12]. In case the samples show that a task thread is stuck in an individual internal method call, it is the backpressure indication for that task. For example,

```

1: procedure backpressureMitigate (Operator backwardOp)
2: backwardOp : backward operator instance
3: node : Node instance
4: task : task instance in the system
5: CPUThreshold : CPU usage threshold
6: inboundThreshold : inbound usage's defined threshold
7: outboundThreshold : outbound usage's defined threshold
8: backpressureThreshold : defined threshold limit for backpressure
9:   for each task in backwardOp do
10:     node ← task.getNode()
11:     if node.cpuUsage < CPUThreshold then
12:       if node.inboundUsage > inboundThreshold ||
13:         node.outboundUsage > outboundThreshold then
14:         inOutOp ← findInOutboundOp()
15:         inOutOp.parallelism = inOutOp.parallelism - 1
16:         newSchedule ← true
17:       else if node.inboundUsage > inboundThreshold
18:         inOp ← findInboundOp()
19:         inOp.parallelism = inOp.parallelism - 1
20:         newSchedule ← true
21:       else
22:         outOp ← findOutboundOp()
23:         outOp.parallelism = outOp.parallelism - 1
24:         newSchedule ← true
25:       end if
26:       break
27:     else
28:       backwardOp.parallelism ← backwardOp.parallelism + 1
29:       newSchedule ← true
30:       break
31:     end if
32:   end for
33:   // alert the user about the backpressure
34:   alert ()
35: end procedure

```

Fig. 3. backpressureMitigate module: get the backward operator, try to mitigate the backpressure for each module, and reduce the parallelism of the operators accordingly.

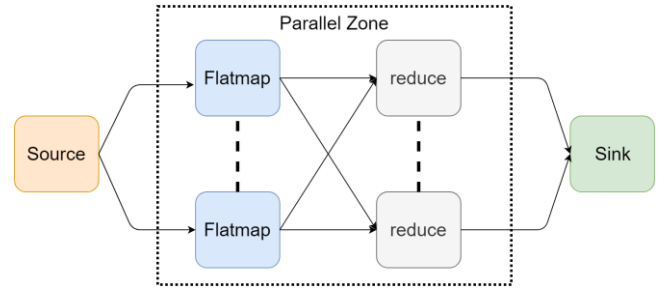


Fig. 5. Evaluation topology: Containing source operator, Flatmap operators, reduce operators, and sink operator.

requesting buffers from the network stack. The dynamic reuse of the backpressure information on the current capacity of the downstream buffer to manage and reschedule or rebalance the task in the task manager might help to divide the data stream into different output streams.

For the verification technique, we simulated a backpressure scenario with a topology of source-flatmap-reduce-sink as shown in Fig. 5. The source operator produces data and transfer is to the flatmap operator instance using the default partitioning scheme. The flatmap operator processed data stream is then reduced through different partitions and finally sink operator receives it for further use. The underlying system configuration is shown in Table 1, we use intel core i-5 system with the CPU power of 3.20 GHz, memory of 12 GB, and x-64 bit based processor. The experimentation is done through a virtualized environment using VMware containerized environment based on Ubuntu 14.04. In the default flink system, the job manager triggers 100 stack traces every 50 ms for each task to decide whether it will cause backpressure or not. It has a defined ratio to show how many of the triggered stack traces were stuck in the internal method calls. For example the value of 0.05 means that five out of hundred calls were stuck in the internal method calls. The defined range of ratios are as bellow,

$0 \leq \text{ratio} \leq 0.10$ { safer }
 $0.10 < \text{ratio} \leq 0.50$ { low risk of backpressure }
 $0.50 < \text{ratio} \leq 1$ { high risk of backpressure }

Table 1
System Configuration

Hardware / Software	Configuration
System	Intel Core i-5
CPU	3.2 GHz
Memory	12GB
Flink	Version 1.2.1
Ubuntu	14.04
Number of instances	2

We introduced backpressure into one of the two nodes to simulate the real-world case where the backpressure propagate from operator to operator. We run six operators on two nodes, and the simulated result for both the default and the

backpressureMitigate strategy is shown in table 2 below. As we can derive from these values that with the forced introduction of backpressure to simulate the real-world like scenario, the ratio for the default is in the safer range only for the source operator, for Flatmap it increases to 0.40, 0.65 for reduce operators, and 0.8 as highest for the sink operator. This effect is due to the fact that the source operator produces records at a higher rate than the sink can consume and the backpressure propagates. The preliminary simulated results verify that the backpressure can be controlled through the mitigation strategy introduced in section III. It reduces the higher backpressure of sink to 0.5, also, decreases the risk of backpressure in case of reduce and Flatmap operators as well.

Table 2

Default-ratio vs. Proposed-ratio values based on the earlier configuration

<i>Subtask</i>	<i>Default-ratio</i>	<i>Proposed-ratio</i>
Source	0.1	0.1
Flatmap	0.4	0.32
Flatmap	0.4	0.36
Reduce	0.65	0.42
Reduce	0.65	0.42
Sink	0.8	0.5

V. CONCLUDING REMARKS

In this article, the design and road towards the mitigation of the backpressure in case of different distributed stream processing systems are discussed. Furthermore, generic back pressure discovery and mitigation algorithm have been proposed to investigate and minimize the effect of back pressure distributed stream processing systems such as Apache

Storm, Flink, IBM STREAM and so on. Since our approach automatically adapts the data flow to the upstream and downstream rate, it has the ability to reduce the bottleneck of the backpressure in DSPS.

REFERENCES

- [1] A. Toshniwal et al., "Apache Storm," Sigmod, 2014.
- [2] M. Zaharia, M. Chowdhury, T. Das, and A. Dave, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," NSDI'12 Proc. 9th USENIX Conf. Networked Syst. Des. Implement., pp. 2–2, 2012.
- [3] S. A. Noghabi et al., "Samza: stateful scalable stream processing at LinkedIn," Proc. VLDB Endow., vol. 10, no. 12, pp. 1634–1645, 2017.
- [4] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness , Latency , and Cost in Massive-Scale , Unbounded , Out-of-Order Data Processing," pp. 1792–1803.
- [5] B. Gedik, H. Andrade, K. L. Wu, P. S. Yu, and M. Doo, "SPADE: The system S declarative stream processing engine," in Proceedings of the ACM SIGMOD International Conference on Management of Data, 2008.
- [6] P. Carbone and S. Ewen, "Apache Flink TM: Stream and Batch Processing in a Single Engine," pp. 28–38.
- [7] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State management in Apache Flink®," Proc. VLDB Endow., 2017.
- [8] H. Luu and H. Luu, "Spark Streaming," in Beginning Apache Spark 2, 2018.
- [9] N. Kruber, "A Deep-Dive into Flink's Network Stack," 2019. [Online]. Available: <https://flink.apache.org/2019/06/05/flink-network-stack.html>. [Accessed: 15-Aug-2019].
- [10] S. Ewen, "Data exchange between tasks in Apache Flink," 2015. [Online]. Available: <https://cwiki.apache.org/confluence/display/FLINK/Data+exchange+between+tasks>.
- [11] B. Lohrmann, P. Janacik, and O. Kao, "Elastic Stream Processing with Latency Guarantees," in Proceedings - International Conference on Distributed Computing Systems, 2015.
- [12] "Monitoring Back Pressure," 2019. [Online]. Available: https://ci.apache.org/projects/flink/flink-docs-stable/monitoring/back_pressure.html#back-pressure. [Accessed: 15-Aug-2019].